

DETECTION OF CYBER ATTACK IN NETWORK USING MACHINE LEARNING TECHNIQUES

PRABHAKARA R UYYALA
COMPUTER AND SCIENCE ENGINEERING
Research Scholar, JS UNIVERSITY,
SIKHOHABAD, UTTAR PRADESH,
Uyyalahome1@gmail.com

ABSTRACT: Contrasted with the past, improvements in PC and correspondence innovations have given broad and propelled changes. The use of new innovations give incredible advantages to people, organizations, and governments, be that as it may, messes some up against them. For instance, the protection of significant data, security of put away information stages, accessibility of information and so forth. Contingent upon these issues, digital fear based oppression is one of the most significant issues in this day and age. Digital fear, which made a great deal of issues people and establishments, has arrived at a level that could undermine open and nation security by different gatherings, for example, criminal association, proficient people and digital activists. Along these lines, Intrusion Detection Systems (IDS) has been created to maintain a strategic distance from digital assaults. Right now, learning the bolster support vector machine (SVM) calculations were utilized to recognize port sweep endeavors dependent on the new CICIDS2017 dataset with 97.80%, 69.79% precision rates were accomplished individually. Rather than SVM we can introduce some other algorithms like random forest, CNN, ANN where these algorithms can acquire accuracies like SVM – 93.29, CNN – 63.52, Random Forest – 99.93, ANN – 99.11.

KEYWORDS: IDS, CNN, ANN, SVM.

I.INTRODUCTION: Today, political and commercial entities are increasingly engaging in sophisticated cyberwarfare to damage, disrupt, or censor information content in computer networks. In designing network protocols, there is a need to ensure reliability against intrusions of powerful attackers that can even control a fraction of parties in the network. The controlled parties can launch both passive (e.g., eavesdropping, nonparticipation) and active attacks (e.g., jamming, message dropping, corruption, and forging). Intrusion detection is the process of dynamically monitoring events occurring in a computer system or network, analysing them for signs of possible incidents and often interdicting the unauthorized access. This is typically accomplished by automatically collecting information from a variety of systems and network sources, and then analysing the information for possible security problems. Traditional intrusion detection and prevention techniques, like firewalls, access control mechanisms, and encryptions,

have several limitations in fully protecting networks and systems from increasingly sophisticated attacks like denial of service. Moreover, most systems built based on such techniques suffer from high false positive and false negative detection rates and the lack of continuously adapting to changing malicious behaviours. In the past decade, however, several Machine Learning (ML) techniques have been applied to the problem of intrusion detection with the hope of improving detection rates and adaptability. These techniques are often used to keep the attack knowledge bases up-to-date and comprehensive. In recent days, cyber-security and protection against numerous cyber-attacks are becoming a burning question. The main reason behind that is the tremendous growth of computer networks and the vast number of relevant applications used by individuals or groups for either personal or commercial use, especially after the acceptance of the Internet of Things (IoT). The cyber-attacks cause severe damage and severe financial losses in large-scale networks. The existing solutions like hardware and software firewalls, user's authentication, and data encryption methods are not sufficient to meet the challenge of upcoming demand, and unfortunately, not able to protect the computer network's several cyber-threats. These conventional security structures are not sufficient as safeguard due to the faster rigorous evolution of intrusion systems. Firewall only controls every access from network to network, which means prevent access between networks. But it does not provide any signal in case of an internal attack. So, it is obvious to develop accurate defense techniques such as machine learning-based intrusion detection system (IDS) for the system's security. In general, an intrusion detection system (IDS) is a system or software that detects infectious activities and violations of policy in a network or system. An IDS identifies the inconsistencies and abnormal behavior on a network during the functioning of daily activities in a network or system used to detect risks or attacks related to network security, like denial-of-service (Dos). An intrusion detection system also helps to locate, decide, and control unauthorized system behaviour such as unauthorized access, or modification and destruction. There are different types of intrusion detection systems based on the user perspective. For instance, they are host based and network-based IDS.

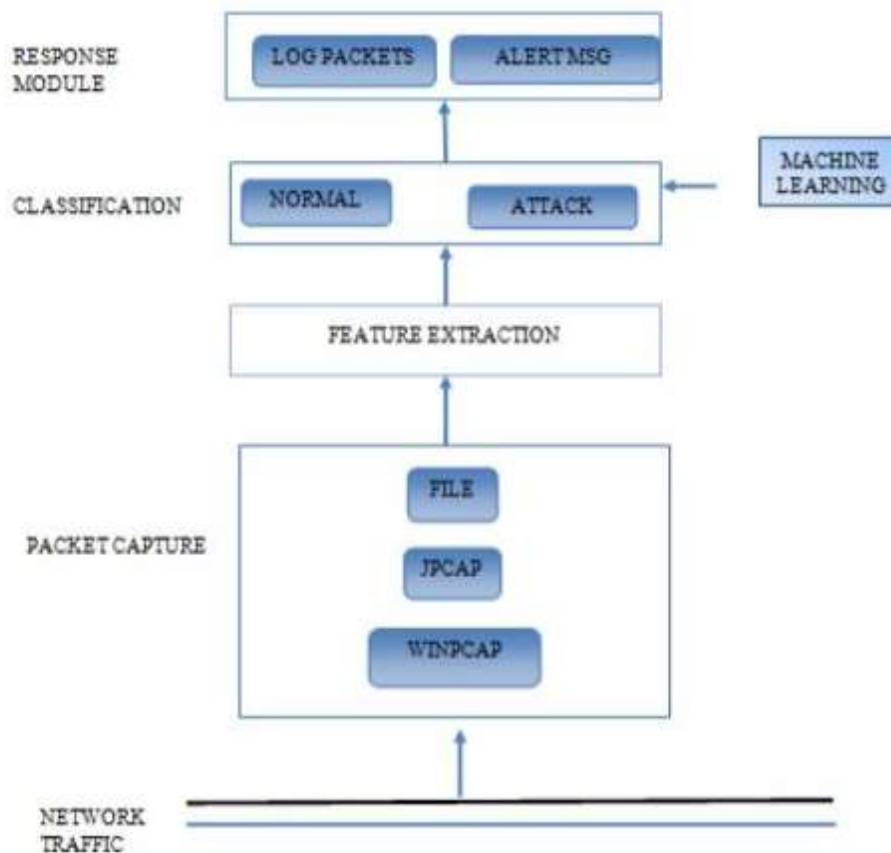
The use of new innovations give incredible advantages to people, organizations, and governments, be that as it may, messes some up against them. For instance, the protection of significant data, security of put away information stages, accessibility of information and so forth. Contingent upon these issues, digital fear based oppression is one of the most significant issues in this day and age. Digital fear, which made a great deal of issues people and

establishments, has arrived at a level that could undermine open and nation security by different gatherings, for example, criminal association, proficient people and digital activists. Along these lines, Intrusion Detection Systems (IDS) has been created to maintain a strategic distance from digital assaults.

II.EXISTING SYSTEM: Blameless Bayes and Principal Component Analysis (PCA) were been used with the KDD99 dataset by Almansob and Lomte [9]. Similarly, PCA, SVM, and KDD99 were used Chithik and Rabbani for IDS [10]. In Aljawarneh et al's. Paper, their assessment and examinations were conveyed reliant on the NSL-KDD dataset for their IDS model [11] Composing inspects show that KDD99 dataset is continually used for IDS [6]–[10]. There are 41 highlights in KDD99 and it was created in 1999. Consequently, KDD99 is old and doesn't give any data about cutting edge new assault types, example, multi day misuses and so forth. In this manner we utilized a cutting-edge and new CICIDS2017 dataset [12] in our investigation.

III.PROPOSED SYSTEM:

Machine Learning algorithms can be used to train and detect if there has been a cyber attack. As soon as the attack is detected, an email notification can be sent to the security engineers or users. Any classification algorithm can be used to categorize if it is a DoS/DDoS attack or not. One example of a classification algorithm is Support Vector Machine (SVM) which is a supervised learning method that analyses data and recognizes patterns. Since we cannot control when, where or how an attack may come our way, and absolute prevention against these cannot be guaranteed yet, our best shot for now is early detection which will help mitigate the risk of irreparable damage such incidents can cause. Organizations can use existing solutions or build their own to detect cyber attacks at a very early stage to minimize the impact. Any system that requires minimal human intervention would be ideal.



3.1 SYSTEM ARCHITECTURE

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

import itertools
import seaborn as sns
import pandas profiling
import statsmodels.formula.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from patsy import dmatrices

/usr/local/lib/python3.6/dist-packages/statsmodels/tools/_testing.py:19: FutureWarning: pandas.util.testing is deprecated. Use the functions in the public API at pandas.testing instead.
import pandas.util.testing as tm

from sklearn import datasets
from sklearn.feature_selection import RFE
import sklearn.metrics as metrics
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2, f_classif, mutual_info_classif

train=pd.read_csv('/content/drive/My Drive/kdd/NSL_Dataset/Train.txt',sep=',')
test=pd.read_csv('/content/drive/My Drive/kdd/NSL_Dataset/Test.txt',sep=',')
  
```

Data preprocessing

```

n [6]: columns=["duration","protocol_type","service","flag","src_bytes","dst_bytes","land",
"wrong_fragment","urgent","hot","num_failed_logins","logged_in",
"num_compromised","root_shell","su_attempted","num_root","num_file_creations",
"num_shells","num_access_files","num_outbound_cmds","is_host_login",
"is_guest_login","count","srv_count","error_rate","srv_error_rate",
"error_rate","srv_error_rate","same_srv_rate","diff_srv_rate","srv_diff_host_rate","dst_host_count","dst_host_sr",
"dst_host_diff_srv_rate","dst_host_same_src_port_rate",
"dst_host_srv_diff_host_rate","dst_host_error_rate","dst_host_srv_error_rate",
"dst_host_error_rate","dst_host_srv_error_rate","attack","last_flag"]

n [7]: train.columns=columns
test.columns=columns

n [8]: train.head()
ut[8]:

```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	root_s
0	0	udp	other	SF	146	0	0	0	0	0	0	0	0	0
1	0	tcp	private	SD	0	0	0	0	0	0	0	0	0	0
2	0	tcp	http	SF	232	8153	0	0	0	0	0	1	0	0
3	0	tcp	http	SF	199	420	0	0	0	0	0	1	0	0
4	0	tcp	private	REJ	0	0	0	0	0	0	0	0	0	0

```

n [9]: test.head()

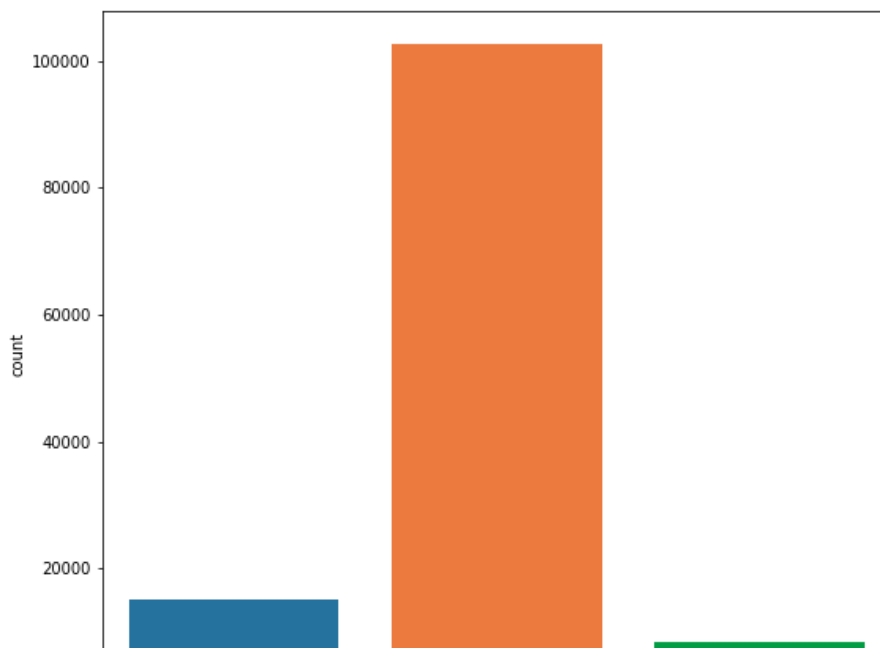
```

Data EDA

```

: # Protocol type distribution
plt.figure(figsize=(9,8))
sns.countplot(x="protocol_type", data=train)
plt.show()

```



Model Building

```
train_X=train_new[cols]
train_y=train_new['attack_class']
test_X=test_new[cols]
test_y=test_new['attack_class']
```

ML Deploy

Logistic Regression

```
# Building Models
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(random_state=0,solver='lbfgs',multi_class='multinomial')
logreg.fit( train_X, train_y)
logreg.predict(train_X)  #by default, it use cut-off as 0.5
```

```
list( zip( cols, logreg.coef_[0] ) )
```

```
logreg.intercept_
```

```
logreg.score(train_X,train_y)
```

Decision Trees

```
train_X.shape
```

```
param_grid = {'max_depth': np.arange(2, 12),
              'max_features': np.arange(10,15)}
```

```
train_y.shape
```

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier, export_graphviz, export
tree = GridSearchCV(DecisionTreeClassifier(), param_grid, cv = 10,verbose=1,n_jobs=-1)
tree.fit( train_X, train_y )
```

```
tree.best_score_
```

```
tree.best_estimator_
tree.best_params_
```

```
train_pred = tree.predict(train_X)
```

```
print(metrics.classification_report(train_y, train_pred))
```

```
test_pred = tree.predict(test_X)
```

Random Forest

```

: from sklearn.ensemble import RandomForestClassifier
  paragrid_rf = {'n_estimators': [50,60,70,80,90,100],
                 'max_features': [2,3,4,5,6,7]}

: from sklearn.model_selection import GridSearchCV
  gscv_rf = GridSearchCV(estimator=RandomForestClassifier(),
                        param_grid=paragrid_rf,
                        cv=10,
                        verbose=True, n_jobs=-1)

  gscv_results = gscv_rf.fit(train_X, train_y)

: gscv_results.best_params_

: gscv_rf.best_score_

: radm_clf = RandomForestClassifier(oob_score=True,n_estimators=80, max_features=5, n_jobs=-1)
  radm_clf.fit( train_X, train_y )

: radm_test_pred = pd.DataFrame( { 'actual': test y,
                                   'predicted': radm_clf.predict( test X ) } )

```

Support Vector Machine (SVM)

```

: from sklearn.svm import LinearSVC
  svm_clf = LinearSVC(random_state=0, tol=1e-5)
  svm_clf.fit(train_X,train_y)

: print(svm_clf.coef_)
  print(svm_clf.intercept_)
  print(svm_clf.predict(train_X))

: from sklearn.svm import SVC
  from sklearn.pipeline import make_pipeline

  model = SVC(kernel='rbf', class_weight='balanced',gamma='scale')

: model.fit(train_X,train_y)

: from sklearn.model_selection import GridSearchCV
  param_grid = {'C': [1, 10],
                'gamma': [0.0001, 0.001]}
  grid = GridSearchCV(model, param_grid)

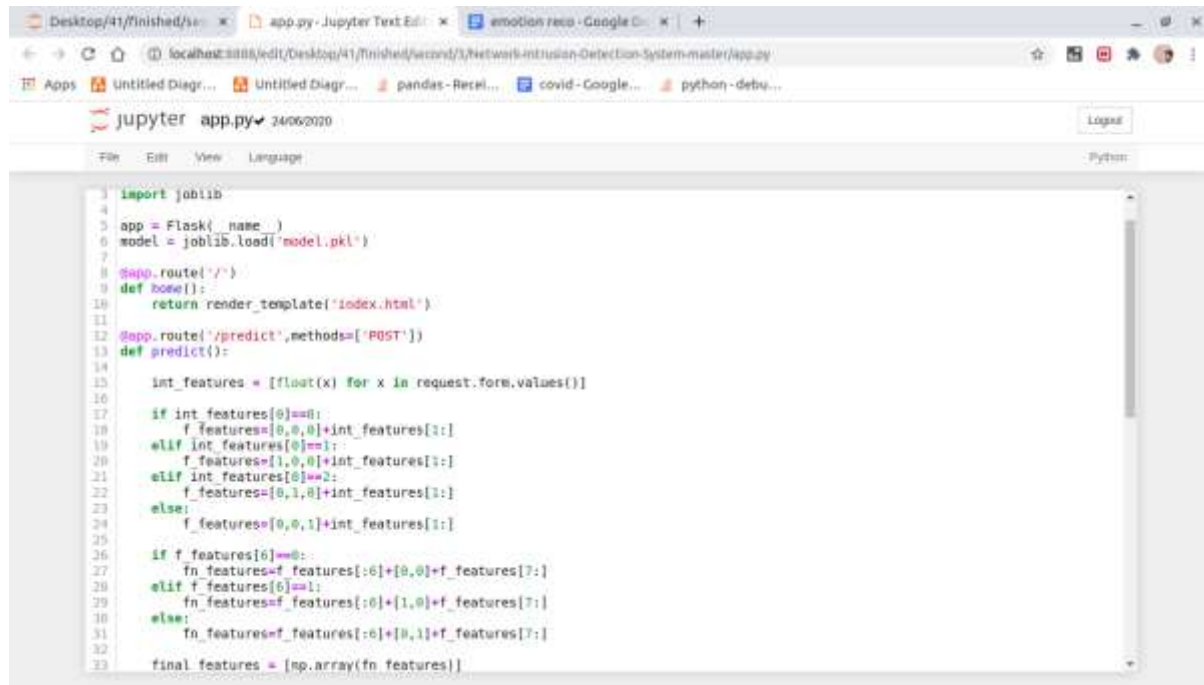
  grid.fit(train_X,train_y)

: print(grid.best_params_)

```

From the score accuracy we concluding the DT & RF give better accuracy and building pickle file for predicting the user input

Application



```

1 import joblib
2
3 app = Flask(__name__)
4 model = joblib.load('model.pkl')
5
6 @app.route('/')
7 def home():
8     return render_template('index.html')
9
10 @app.route('/predict', methods=['POST'])
11 def predict():
12     int_features = [float(x) for x in request.form.values()]
13
14     if int_features[0]==0:
15         f_features=[0,0,0]+int_features[1:]
16     elif int_features[0]==1:
17         f_features=[1,0,0]+int_features[1:]
18     elif int_features[0]==2:
19         f_features=[0,1,0]+int_features[1:]
20     else:
21         f_features=[0,0,1]+int_features[1:]
22
23     if f_features[6]==0:
24         fn_features=f_features[:6]+[0,0]+f_features[7:]
25     elif f_features[6]==1:
26         fn_features=f_features[:6]+[1,0]+f_features[7:]
27     else:
28         fn_features=f_features[:6]+[0,1]+f_features[7:]
29
30 final_features = np.array(fn_features)
31

```

Localhost - in cmd python app.py

```

user@ramesh:~/Desktop/41/finished/second/3/Network-Intrusion-Detection-System-master$ python3 app.py
/home/user/.local/lib/python3.6/site-packages/sklearn/base.py:334: UserWarning:
Trying to unpickle estimator LogisticRegression from version 0.22.1 when using v
ersion 0.23.2. This might lead to breaking code or invalid results. Use at your
own risk.
  UserWarning)
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

```


Network Intrusion Detection System

Attack:
Other

Number of connections to the same destination host as the current connection in the past two seconds :
10000

The percentage of connections that were to different services, among the connections aggregated in dst_host_count :
dst_host_srv_count

The percentage of connections that were to the same source port, among the connections aggregated in dst_host_srv_count :
dst_host_srcport_dst_host_srv

The percentage of connections that were to the same service, among the connections aggregated in dst_host_count :
dst_host_srcport_dst_host_srv

Number of connections having the same port number :
dst_host_srv_count

Enter the input

Network Intrusion Detection System

Status of the connection - Normal or Error :
Other

Last Flag :
3

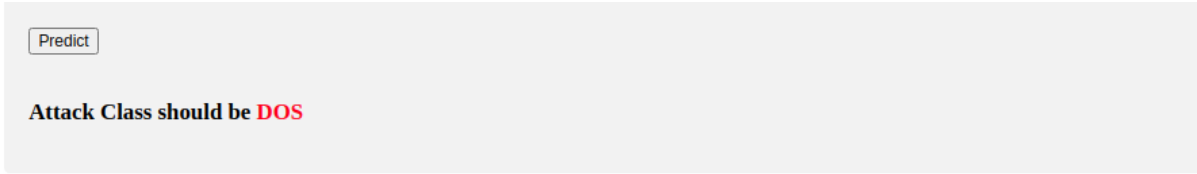
1 if successfully logged in; 0 otherwise :
5

The percentage of connections that were to the same service, among the connections aggregated in count :
5

The percentage of connections that have activated the flag (4) s0, s1, s2 or s3, among the connections aggregated in count :
11

Destination network service used http or not :
No

Predict

Predict attack -

 A screenshot of a web application interface. At the top, there is a button labeled 'Predict'. Below the button, a message is displayed: 'Attack Class should be DOS', where 'DOS' is highlighted in red text.

Attack Class should be **DOS**

CONCLUSION: Right now, estimations of help vector machine, ANN, CNN, Random Forest and profound learning calculations dependent on modern CICIDS2017 dataset were introduced relatively. Results show that the profound learning calculation performed fundamentally preferable outcomes over SVM, ANN, RF and CNN. We are going to utilize port sweep endeavors as well as other assault types with AI and profound learning calculations, apache Hadoop and sparkle innovations together dependent on this dataset later on. All these calculation helps us to detect the cyber attack in network. It happens in the way that when we consider long back years there may be so many attacks happened so when these attacks are recognized then the features at which values these attacks are happening will be stored in some datasets. So by using these datasets we are going to predict whether cyber attack is done or not. These predictions can be done by four algorithms like SVM, ANN, RF, CNN this paper helps to identify which algorithm predicts the best accuracy rates which helps to predict best results to identify the cyber attacks happened or not.

REFERENCES

- [1] K. Graves, Ceh: Official certified ethical hacker review guide: Exam 312-50. John Wiley & Sons, 2007.
- [2] R. Christopher, “Port scanning techniques and the defense against them,” SANS Institute, 2001.
- [3] M. Baykara, R. Das,, and I. Karado ğan, “Bilgi g ğuvenli ğisistemlerindekullanılanarac,larinincelenmesi,” in 1st International Symposium on Digital Forensics and Security (ISDFS13), 2013, pp. 231–239.
- [4] S. Staniford, J. A. Hoagland, and J. M. McAlerney, “Practical automated detection of stealthy portscans,” Journal of Computer Security, vol. 10, no. 1-2, pp. 105–136, 2002.
- [5] S. Robertson, E. V. Siegel, M. Miller, and S. J. Stolfo, “Surveillance detection in high bandwidth environments,” in DARPA Information Survivability Conference and Exposition, 2003.Proceedings, vol. 1. IEEE, 2003, pp. 130–138.

- [6] K. Ibrahimi and M. Ouaddane, "Management of intrusion detection systems based-kdd99: Analysis with lda and pca," in Wireless Networks and Mobile Communications (WINCOM), 2017 International Conference on. IEEE, 2017, pp. 1–6.
- [7] N. Moustafa and J. Slay, "The significant features of the unsw-nb15 and the kdd99 data sets for network intrusion detection systems," in Building Analysis Datasets and Gathering Experience Returns for Security (BADGERS), 2015 4th International Workshop on. IEEE, 2015, pp. 25–31.
- [8] L. Sun, T. Anthony, H. Z. Xia, J. Chen, X. Huang, and Y. Zhang, "Detection and classification of malicious patterns in network traffic using benford's law," in Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), 2017. IEEE, 2017, pp. 864–872.
- [9] S. M. Almansob and S. S. Lomte, "Addressing challenges for intrusion detection system using naive bayes and pca algorithm," in Convergence in Technology (I2CT), 2017 2nd International Conference for. IEEE, 2017, pp. 565–568.
- [10] M. C. Raja and M. M. A. Rabbani, "Combined analysis of support vector machine and principle component analysis for ids," in IEEE International Conference on Communication and Electronics Systems, 2016, pp. 1–5.